



BloodChain: Smart-Contract-Enforced Vein-to-Vein Blood Custody on Hedera

[Author 1], [Author 2], [Author 3], [Author 4], [Author 5] (University of Zurich)

Summer School: Deep Dive into Blockchain 2026

21 July 2026

ABSTRACT

Blood custody is a multi-organization process with no natural owner of the record: collection centers, testing laboratories, transporters, and hospitals each keep self-serving logs, and the safety rule that no unit may be released without a passing infectious-disease test is enforced only by policy. BloodChain implements vein-to-vein custody tracking on the public Hedera network, combining a non-fungible token per physical blood unit (Hedera Token Service), a consensus-ordered event log (Hedera Consensus Service), and two Solidity smart contracts that turn the release rule and an anti-fraud oversight regime into network-enforced logic. A deliberately failing demonstration shows a contaminated unit being physically blocked from transfer and its entire donation batch quarantined in seconds, including a sibling unit that had already changed hands. An economic oversight layer adds bonded participation, capped slashing, hash-based staff traceability, and a reputation-weighted authority election. The full system, including a React application whose every displayed event links to the public ledger, was deployed and verified end to end on Hedera testnet. We report the design, a live evaluation, and an explicit account of the trust boundaries a ledger cannot cross.

1. INTRODUCTION

A unit of donated blood passes through at least four organizations on its way from a donor's vein to a patient's vein. Each hand-off crosses an administrative boundary: the collection center, the testing laboratory, the logistics provider, and the hospital all operate separate record systems with different software and different incentives. Four practical failure modes motivate this project. First, the central safety rule — no unit may be released without a passing test result — is enforced by procedure rather than by mechanism, and a skipped step in one organization's workflow is invisible to every other participant. Second, recalls are slow: one donation is separated into several sibling products, and when one is found contaminated, locating the others is a manual, cross-organizational search measured in hours or days. Third, blood diversion — a unit leaving the legitimate chain for informal resale — leaves at most an internal discrepancy in records controlled by the very party that diverted it. Fourth, when investigations do happen, each party's records are self-serving evidence that their owner can quietly amend.

These four failure modes share a root cause: some single party always controls the record. That observation, rather than enthusiasm for the technology, is our argument for a public distributed ledger. A shared database would fix fragmentation but not control, because someone must administer it. A public ledger has no administrator to trust: its history is append-only by construction, its ordering is fixed by network consensus, and rules deployed as smart contracts execute identically for every participant and cannot be skipped by any of them.

What we believe is distinctive in our approach is threefold. (1) The smart contract is an enforcement point, not a notary: the token representing a blood unit physically cannot move until the contract verifies a recorded passing

test, and our demonstration deliberately attempts the forbidden transfer to show the network refusing it. (2) Fraud detection is built on the absence of events: accepting that a ledger cannot witness a physical hand-off, the system watches for the closing event that should exist and treats silence beyond a holding window as the detectable signature of diversion, feeding an on-chain investigation and penalty pipeline. (3) Verifiability is treated as the product surface: every event shown in our application links to the public Hedera explorer, custody trails are read by the browser directly from public infrastructure rather than from our own servers, and each unit receives a QR code binding the physical bag to its public history page.

The remainder of this report describes the platform background (Section 2), the system design across the token, log, contract, and application layers (Section 3), a live evaluation on Hedera testnet (Section 4), and a discussion of case selection, challenges, significance, and generalization (Section 5), followed by a summary.

2. BACKGROUND: THE HEDERA PLATFORM

Hedera is a public distributed ledger whose consensus is based on a hashgraph rather than a chain of blocks (Baird, 2016). Nodes continuously exchange state with randomly selected peers, including the history of the exchanges themselves; from this shared communication graph, every node can compute how every other node would vote on transaction order without votes being sent. The practical consequences that matter for this project are fast finality (a few seconds, with no forks to out-wait), asynchronous Byzantine fault tolerance, fees fixed in fiat terms and small enough to log every custody event individually, and a consensus timestamp: a network-agreed, nanosecond-resolution time assigned to every transaction, which gives any two custody events a definite order that no participant can influence.

Three Hedera services divide the work. The Hedera Token Service (HTS) manages tokens as native network objects; we use a non-fungible token (NFT) class, in which each minted instance carries a unique network-assigned serial number, so that one NFT represents exactly one physical blood unit and token ownership answers custody questions at the network level. HTS enforces role keys natively: a supply key gates minting and burning, and a wipe key allows the platform to retire a unit from any holder. The Hedera Consensus Service (HCS) provides append-only, consensus-ordered message topics; our topic is the audit trail, one JSON message per custody event. The Hedera Smart Contract Service executes Solidity contracts on the Ethereum Virtual Machine (EVM), identifying callers by cryptographically verified addresses (`msg.sender`), which our access control builds on. Finally, mirror nodes replay finalized history and expose it as a free public REST interface, on which both our audit tooling and the public explorer HashScan are built. All experiments run on testnet, a functionally identical network with free currency; our team accounts use ECDSA (secp256k1) keys, which map naturally to EVM addresses inside contracts.

3. SYSTEM DESIGN AND IMPLEMENTATION

3.1 Architecture overview

The system is layered. On-chain: the BLOOD NFT class (one serial per unit), one HCS topic (all custody events), and two Solidity contracts — `BloodUnitGate`, holding the release rule, and `BloodOversight`, holding the anti-fraud regime. Off-chain: a Node.js core exposing one function per custody action; a local JSON index that caches unit state and is explicitly not a source of truth (it can be rebuilt from the topic's full history by a replay tool, and a read-only audit mode diffs the cache against chain history to detect drift); a thin localhost API server that holds all private keys and signs each action as the correct party; and a React frontend. The frontend's write path goes through the API server, but its read path deliberately does not: custody trails and event feeds are fetched by the browser directly from the public mirror node, and every displayed event links to its transaction on HashScan, so a reader can verify any claim on infrastructure the team does not control.

3.2 The custody core

A unit's life is five functions. `mintUnit()` turns a collection-time barcode scan into an NFT carrying only the donor batch and collection center (HTS caps NFT metadata at 100 bytes; full detail belongs to the log) and writes a `COLLECTED` event. `submitTestResult()` records the laboratory's verdict twice: on the gate contract, where enforcement lives, and on the topic, where the audit trail lives. Critically, the contract call is signed by the laboratory's own key: the contract's `onlyAuthorizedLab` modifier verifies `msg.sender` against an allow-list maintained by the contract owner, so a forged verdict from an unauthorized account is rejected by the network

itself, and the emitted event permanently attributes each result to the attesting laboratory. `transferCustody()` is the enforcement point: before any token moves, it calls the gate's `requireClearance(serial)`, a function that reverts unless a Passed verdict is recorded for that serial. On revert, the transfer is never attempted; a `TRANSFER_BLOCKED` event makes the refusal part of the permanent record. Only a genuine contract revert is treated as a block — infrastructure errors are surfaced to the caller instead of being written into the audit trail as false fraud signals. `closeUnit()` burns the NFT at end of life (transfusion or disposal) using the wipe key, so the set of live tokens equals the set of blood in circulation. `flagBatch()` is the recall: given one bad unit, it quarantines every sibling of the same donation in one sweep — including siblings already transferred away, and, for siblings already transfused, an explicit post-use alert for patient follow-up.

3.3 The oversight layer

The oversight layer answers what happens when an authorized participant misbehaves, under an honestly stated threat model: a ledger cannot see a physical hand-off, but it can see the absence of an event that should exist. Every legitimate unit ends with a transfusion or disposal event inside a holding window (ten days by default); a stale-unit monitor flags any unit held in silence past the window with a permanent `STALE_ALERT`. Organizations register on the `BloodOversight` contract by posting a 10 HBAR bond from their own account — collateral that converts punishment from an entry in a database into an automatic economic consequence. Any active member may open an investigation (detection is not a privilege); the elected authority delivers the verdict. Slashing is graduated by design: a single verdict can never take more than 20% of the remaining bond, and suspension requires a pattern — five guilty verdicts or a bond below a quarter of the minimum — because removing a hospital from the blood supply over one finding would harm patients more than the fraud did. A rehabilitation path (bond top-up plus a discretionary reinstatement that forgives one scandal) exists for the same reason. Staff traceability records the nurse or technician for every test, on-chain only as a SHA-256 hash — attribution without personal data — and the authority can suspend a hash, making future results tagged with it rejectable.

The oversight authority itself is elected by the registered organizations, with weight computed by the contract at the moment of voting: $\text{weight} = 10 + 2 \cdot \text{tenure}(\text{months}) + \text{reviewScore}/10 - 2 \cdot \text{scandals}$, floored at one for active members and zero for suspended ones. Each term is an argument: the base keeps new members enfranchised, tenure rewards sustained participation, review scores import external reputation, and the scandal malus makes fraud politically expensive in the very next election. The handover is real: after `closeElection()`, the previous authority's privileged calls revert.

3.4 Application layer

Because private keys cannot live safely in a browser, all state-changing actions pass through a localhost API server that signs as the correct party (the laboratory signs verdicts, the current holder signs transfers, each organization signs its own oversight actions). The React frontend provides role dashboards (blood bank, laboratory, hospital/transport, oversight authority), a live custody-flow visualization in which units move between stations as their on-chain status changes and blocked transfers visibly bounce off the gate, a public per-unit trace view with a printable QR code linking the physical bag to its history page, and a reconciliation view that cross-checks on-chain transfusion events against an independent (mocked) patient-record extract — demonstrating the countermeasure to a faked closing event. When no credentials are configured the application runs against an in-memory simulation of identical rules under a prominent `SIMULATION` badge; it never presents simulated data as ledger-backed.

4. EVALUATION ON HEDERA TESTNET

The complete system was deployed to Hedera testnet and exercised end to end. Table 1 lists the deployed artifacts; all are publicly inspectable on hashscan.io/testnet by identifier.

Artifact	Identifier	Role
BLOOD token (HTS, NFT)	0.0.9663839	One serial per physical blood unit
Custody topic (HCS)	0.0.9663848	Append-only event log, one JSON message per event
BloodUnitGate (contract)	0.0.9663858	Release rule: <code>requireClearance()</code> reverts without a passing test

BloodOversight (contract)	0.0.9663881	Bonds, investigations, slashing, staff registry, elections
Operator / blood bank	0.0.9628733	Treasury; token supply and wipe keys
Laboratory account	0.0.9637911	Authorized on the gate; signs all test verdicts
Hospital / transport accounts	0.0.9638100 / 0.0.9642712	Custody parties; bonded oversight members

Table 1. Live testnet deployment (all identifiers publicly verifiable on hashscan.io/testnet).

Three verification stages were run against this deployment. The core demonstration minted unit A, recorded a passing verdict signed by the laboratory's own key, and transferred the unit; it then minted sibling unit B from the same donation, recorded a failing verdict, and attempted the forbidden transfer. The gate contract reverted, the token did not move, and the refusal was logged as a permanent TRANSFER_BLOCKED event; flagging unit B then quarantined unit A at its new holder within seconds. The oversight demonstration registered all four organizations with real bonds, let a transferred unit go silent past a compressed holding window, and drove the full pipeline: stale alert, investigation opened by an ordinary member, guilty verdict slashing the hospital's bond from 10 to 8 HBAR (exactly the 20% cap) with one scandal recorded and no suspension — the graduated design behaving as specified — followed by an election in which the freshly penalized hospital voted with visibly reduced weight and authority genuinely transferred to the laboratory. Finally, the automated integration suite (minting, gate blocking, batch recall) passed 3/3 against testnet, and the cache audit rebuilt unit state from the topic's 36 consensus-ordered messages and found zero drift against the local index. Table 2 summarizes.

Verification	Outcome
Core custody demo (gate block + batch recall)	Pass — forbidden transfer reverted; sibling quarantined after hand-off
Oversight demo (stale, slash, election)	Pass — bond 10 → 8 HBAR (20% cap); authority handed to laboratory
Integration tests vs. testnet	3/3 pass
Cache-vs-ledger drift audit	36 on-chain messages, 13 units, no drift
Frontend live mode	Real mirror-node feed; per-event HashScan links resolve

Table 2. End-to-end verification results on the live deployment (21 July 2026).

5. DISCUSSION

5.1 Why this case

We chose blood custody because its trust problem is structural rather than technological: the participants genuinely cannot agree on a record keeper, which is the precondition under which a public ledger outperforms a database. The central safety rule is binary and mechanically checkable, making it an ideal candidate for contract enforcement at a single choke point (the token transfer). The stakes justify per-unit overhead in a way consumer-goods tracking does not. And the scope is honest for a course project: five custody functions, two contracts, one topic — small enough to build correctly, rich enough to surface real problems.

5.2 Challenges and limitations

We consider the explicit trust boundaries as important as the working features. The oracle problem is the deepest: the ledger proves a record was never altered after writing, not that it was true when written — a laboratory can record a wrong verdict and the gate will faithfully enforce it. Our mitigation is attribution rather than omniscience: verdicts are signed by authorized laboratory keys and tied to registered staff hashes, so wrongness is never anonymous; production would add independent re-testing and random audits. The diversion gap follows the same pattern: a hospital that fabricates a transfusion event defeats the stale-unit monitor, and only reconciliation against an independent record system — demonstrated in the application with mocked patient records — can catch it. Physical-digital binding remains procedural: a QR code ties a bag to a history page, but someone must scan honestly. Governance carries known compromises: review scores are written by the authority acting as an oracle; the incumbent could stall an election (a deadline-based close is the known fix); investigations are decided by one authority rather than a panel; and our four organizations are testnet accounts controlled by one team, demonstrating mechanism rather than politics. Engineering-level gaps are recorded in the repository's audit notes: bonds currently have no withdrawal path, laboratory authorization is not revocable,

an authorized laboratory can overwrite its own verdict (detectably, but not preventably), and the two contracts do not yet consult each other's state.

5.3 Academic and social significance

Academically, the project is a case study in hybrid enforcement: a worked separation of the guarantees that can move on-chain (custody gating, economic slashing, weighted governance) from those that are irreducibly social (verdict truth, physical hand-offs). Two mechanisms seem transferable beyond this domain: absence-of-event detection, which derives a fraud signal from what a ledger cannot see, and graduated slashing with a rehabilitation path, which acknowledges that critical-infrastructure participants cannot simply be ejected. Socially, the demonstrated properties target documented weaknesses of multi-party blood supply chains — recalls measured in days, diversion hidden inside single-party records — in precisely the low-mutual-trust settings where a neutral shared record is otherwise unobtainable (World Health Organization, 2024). We also regard the project's insistence on honest claims as a social contribution in itself: overclaiming what a ledger guarantees erodes exactly the trust such systems are meant to create.

5.4 Generalization to other domains

The reusable core is a composite pattern: a per-item token, an append-only event log, a contract gate on a binary safety predicate, absence-of-event monitoring, and bonded, elected oversight. It maps with little modification onto organ and tissue transport (tighter deadlines), pharmaceutical cold chains (the gate predicate becomes a temperature attestation; the batch recall becomes a lot recall), forensic evidence chains of custody (where a gap in transfers is precisely the alarm courts need), food-safety recalls, and humanitarian aid distribution (bonded local distributors, diversion detected as undelivered silence). The limitation transfers equally: in every derivation, the last physical mile still requires reconciliation against an independent source.

6. REPRODUCIBILITY AND OPEN MATERIAL

Everything reported above is independently checkable. The repository contains the full source (contracts, custody core, oversight tooling, API server, and frontend), the setup scripts in run order, and four companion documents: a zero-prior-knowledge technical explainer, a step-by-step team runbook, the security audit notes underlying Section 5.2, and the course discussion answers. The live deployment identifiers in Table 1 resolve on the public explorer: opening hashscan.io/testnet/topic/0.0.9663848 shows every custody event of this evaluation with its consensus timestamp, and each unit's full history can be replayed from the public mirror node without any account, key, or cooperation from the team.

The application itself supports keyless examination: without credentials it runs against an in-memory simulation of the identical rules under an explicit SIMULATION badge, including a scripted end-to-end story mode used in the presentation; with the shared testnet identifiers configured, all read paths are live against the public network. A per-unit QR code links each (hypothetical) physical bag to its public trace page, closing the loop between the artifact a grader can hold and the ledger entry anyone can verify.

7. SUMMARY

BloodChain demonstrates, on a live public network, that the riskiest claims of ledger-based custody tracking can be made real rather than rhetorical: a smart contract that physically prevents an untested or failed blood unit from moving, a batch recall that reaches already-transferred siblings in seconds, and an oversight regime in which fraud is detected from silence, punished economically in a graduated way, and remembered by a reputation-weighted electorate. The system's read path is fully public — every event verifiable by anyone on infrastructure the team does not control — and its documentation states plainly which guarantees the ledger provides and which must remain social. We regard that explicit boundary, together with the working end-to-end deployment, as the project's main result.

8. AUTHOR CONTRIBUTIONS

All authors conceived and designed the project. [Initials] implemented the custody core and the smart contracts. [Initials] developed the oversight layer and the demonstration scenarios. [Initials] built the API server and frontend application. [Initials] performed the testnet deployment and verification. [Initials] wrote the

documentation and this report. All authors revised and accepted the final version of this document. [Replace with the actual distribution before submission.]

9. REFERENCES

- Baird, L. (2016). The Swirlds hashgraph consensus algorithm: Fair, fast, Byzantine fault tolerance (Swirlds Tech Report SWIRLDS-TR-2016-01). Swirlds, Inc.
- Buterin, V. (2014). Ethereum: A next-generation smart contract and decentralized application platform [White paper]. <https://ethereum.org/whitepaper>
- Hedera. (2026). Hedera documentation: Token Service, Consensus Service, Smart Contract Service, mirror nodes. <https://docs.hedera.com>
- Nakamoto, S. (2008). Bitcoin: A peer-to-peer electronic cash system [White paper]. <https://bitcoin.org/bitcoin.pdf>
- Tian, F. (2016). An agri-food supply chain traceability system for China based on RFID & blockchain technology. In Proceedings of the 13th International Conference on Service Systems and Service Management (ICSSSM) (pp. 1–6). IEEE.
- World Health Organization. (2024). Blood safety and availability [Fact sheet]. <https://www.who.int/news-room/fact-sheets/detail/blood-safety-and-availability>